MARL Architecture Blueprint — APSM-MTP Phase 3/4

Decentralized Edge Workload Forecasting + Autonomous Resource Orchestration

Status: Design-only. No code has been written or modified. **Basis:** Direct inspection of the repository at `d:\coolage study materials\MTP` **Date:** 2026-10-22

---

1. Executive Recommendation

The existing APSM system is a complete, verified Phase 1/2 codebase. The recommended Phase 3/4 extension is a **strictly additive, modular MARL task-offloading layer** that reads from—but never modifies—the existing gossip simulation infrastructure.

Summary of key decisions:

Decision	Recommendation
Dataset (APSM experiments)	Porto Taxi synthetic — unchanged
Dataset (MARL experiments)	Alibaba Cluster Trace 2017
MARL algorithm	MAPPO with CTDE
Agent architecture	Homogeneous shared-parameter actor MLP [64, 64]
Training arrangement	Offline centralized training, frozen decentralized inference
Reward structure	Local + global blend (λ tuned on validation set)
Task execution model	Trace-calibrated discrete-event simulation
HRAC	Not adopted for Phase 3
LGRS	Minimal adoption — centralized critic during training only
Minimum experiment set	E1, E2 (existing), E3, E4, A1 (heuristic), A2 (local reward only)
Scaling	N ∈ {10, 50} for Phase 3; N=100 pending feasibility check

> *[!CAUTION] Do not begin implementation until you have (1) confirmed Alibaba 2017 `batch_task.csv` duration field validity, (2) received supervisor sign-off on this blueprint, and (3) answered the 6 unresolved questions in Section 18.*

---

2. Verified Description of the Existing APSM Architecture

All facts below are confirmed by direct source inspection. Every claim cites an exact file and line.

2.1 LSTM Forecasting Model

File: src/utils/model_creators.py (file:///d:/coolage%20study%20materials/MTP/src/utils/model_creators.py)

```
Input shape:   (batch, 4 timesteps, n_input_features)
Architecture:  LSTM(50, tanh) → LSTM(50, tanh) → Dropout(0.2)
               → Dense(32, relu) → Dropout(0.2) → Dense(n_output_vars, relu)
Optimizer:     Adam(lr=0.001, epsilon=1e-6)
Loss:          MSE per output variable
n_input_features = 9  (1 traffic value + 8 binary-encoded time features)
n_output_vars  = 1    (single-step-ahead workload forecast)
```

The prediction is a scalar workload value, MinMax-normalized ∈ [0, 1].

Critical finding: The forecast $\hat{y}(t+1)$ is **not exposed to any external module**. It is used only internally within `node.perform_update()` → `node.train_model()` to produce `val_loss`, which then feeds the APSM filter. A new read-only interface must be designed to expose it to the MARL layer without modifying `node.py`.

2.2 APSM Semantic Filter

Files: src/gossiplearning/node.py (file:///d:/coolage%20study%20materials/MTP/src/gossiplearning/node.py) L101–L189, src/gossiplearning/event.py (file:///d:/coolage%20study%20materials/MTP/src/gossiplearning/event.py) L159–L173

Verified execution sequence:

Step	File:Line	Operation
Training completes	`event.py` process_receive_model_event	`node.perform_update()` returns `val_loss`
APSM state update	`event.py` L335	`node.update_semantic_state(val_loss)`
Surprise score	`node.py` L143	$\varepsilon(t)$ = `abs(val_loss - self._best_val_loss)`
Threshold compute	`node.py` L151	$\tau(t)$ = `semantic_k × std(error_window[-N:])`
Gating decision	`event.py` L162	`node.should_suppress_transmission()`
Heartbeat safety	`node.py` L182	Force transmit if `consecutive_suppressions >= semantic_heartbeat`

Verified active parameter values (from src/experiment_config.py (file:///d:/coolage%20study%20materials/MTP/src/experiment_config.py)):

- `semantic_k = 2.0`
- `semantic_window = 50` (deque fills before gating activates)
- `semantic_heartbeat = 5` (config.py default is 3, overridden to 5 in experiment_config.py line 67)

> *[!NOTE] **Heartbeat discrepancy:** The handoff document states H=5. `config.py` line 178 sets the default to **3**. `experiment_config.py` line 67 overrides it to **5**. The actual experimental value is 5, matching the handoff. Document this in the thesis to avoid confusion.*

The following public attributes on `Node` are available for the MARL state interface without modifying `node.py`: `node.last_surprise`, `node.last_threshold`, `node.consecutive_suppressions`, `node.suppressed_count`, `node._best_val_loss` (accessible, prefixed with underscore but not name-mangled).

2.3 Discrete-Event Simulator

File: src/gossiplearning/simulator.py (file:///d:/coolage%20study%20materials/MTP/src/gossiplearning/simulator.py)

Uses Python `PriorityQueue` over `dataclass Event` objects sorted by `time: int` (discrete seconds). Event types: `SendModelsLoopEvent`, `ReceiveModelEvent`, `SaveModelEvent`, `FailedNodeEvent`, `RecoveryNodeEvent`.

Critical finding: The simulator contains **zero concepts of tasks, CPU load, memory, queues, or execution time**. `TRAIN_TIME = 5` (event.py line 57) is a hardcoded constant. Transmission time is `random.randint(25, 35)` seconds.

Conclusion: The current simulator cannot model task offloading. A separate, fully decoupled task-execution simulator is required.

2.4 Dataset Pipeline

Files: src/2_dataset_generation.py (file:///d:/coolage%20study%20materials/MTP/src/2_dataset_generation.py), src/utils/data.py (file:///d:/coolage%20study%20materials/MTP/src/utils/data.py)

The workload data is **not real taxi GPS observations**. It is an AR(4) noise signal overlaid on a smooth day-of-week/time-of-day trend, anchored to real antenna tower coordinates from `porto_cells.parquet`. There are no CPU, memory, task duration, or queue fields anywhere in the dataset. Input window: 4 × 15-minute steps.

2.5 History and Metrics

File: src/gossiplearning/history.py (file:///d:/coolage%20study%20materials/MTP/src/gossiplearning/history.py)

Current `History` dataclass fields: `messages`, `trainings`, `stopped_time`, `nodes_training_history`, `nodes_test_history`, `nodes_failures_history`, `suppressed_packets` (Phase 2 addition), `semantic_surprise_scores` (Phase 2 addition). No task-level, queue-level, or resource-level fields exist.

---

3. Research Gap and Proposed Contribution

Verified Gap

The APSM pipeline outputs: a trained LSTM model, a per-round suppress/transmit decision, and an internal validation loss trajectory. It does **not** expose the workload forecast externally, model CPU/memory, route computational tasks, or respond proactively to predicted overload.

Proposed Contribution

> ***Research Question:*** *Can workload forecasts from a gossip-trained LSTM, operating under APSM communication constraints, serve as actionable intelligence for a MARL policy to proactively offload tasks and reduce deadline-miss rates in a fully decentralized P2P edge mesh?*

This is distinct from existing work on three axes:

1. Forecast-driven MARL offloading (Liu et al. 2020; Chen et al. 2021) assumes centralized coordination or full observability — this work operates under strict partial observability in a gossip mesh.
2. APSM creates **intentional information sparsity** — the interaction between semantic communication gating and MARL state freshness is an unexplored research question.
3. No prior work combines adaptive gossip communication gating with decentralized MARL task scheduling in a k-NN P2P topology.

---

4. Relationship Between the Two Base Papers

Dimension	Tundo et al. 2025	Yao et al. 2025	This Thesis
Environment	P2P edge mesh	Cloud cluster	P2P edge mesh
Learning objective	Gossip FL (model weights)	MARL (scheduling policy)	Both, layered
Topology	k-NN random graph	Hierarchical datacenter	k-NN random graph
Resource action	None (passive)	CPU/memory allocation	Task offload / local exec

Dimension	Tundo et al. 2025	Yao et al. 2025	This Thesis
Communication	All-round gossip	Centralized scheduler	APSM-gated gossip

Adaptation decisions:

- **HRAC:** Not adopted for Phase 3. All nodes are structurally homogeneous in a k-NN topology. Revisit in Phase 4 only if ablation results motivate it.
- **LGRS:** Partially adopted — centralized critic during offline training only; no persistent central reward service at runtime.

---

5. Architecture Diagram

flowchart TD subgraph EX["EXISTING SYSTEM — Phase 1 and 2 — DO NOT MODIFY"] A["Porto Taxi\nSynthetic Time-Series"] --> B["Node NPZ datasets\nX_train, Y_train, X_val, Y_val, X_test, Y_test"] B --> C["2-Layer LSTM\n50-50 units, tanh, Dropout 0.2\nAdam lr=0.001"] C -->|val_loss| D["APSM Semantic Filter\nepsilon = abs(val_loss - best_val_loss)\ntau = 2 * std(window of 50)"] D -->|suppress or transmit| E["P2P Gossip Mesh\nAge-Weighted Merge\nk=3 neighbors\nHeartbeat H=5"] E -->|merged weights| C C -->|model.h5| F["history.json\nMSE, packets, suppressed_packets\nsemantic_surprise_scores"] end subgraph PR["PROPOSED EXTENSION — Phase 3 and 4 — NEW MODULES ONLY"] G["Forecast State Interface\nmarl/state_interface.py\nReads y_hat, val_loss, APSM state\nOutputs NodeObservation"] H["Task Execution Simulator\nmarl/task_env.py\nAlibaba 2017 trace\nQueues, CPU util, deadlines"] I["MARL Actor Policy\nmarl/policy.py\nShared-param MLP 64-64\nAction: local, offload_j, defer"] J["Centralized Critic\nmarl/critic.py\nTRAINING ONLY — discarded after\nObserves global state of all N nodes"] K["MARL Metrics Logger\nmarl/metrics.py\nLatency, miss-rate, utilization"] end C -->|"expose y_hat — NEW INTERFACE"| G D -->|"APSM state epsilon tau suppressed"| G G --> I H --> I I -->|action| H H -->|reward| I H -->|reward| J J -->|critic gradient| I I --> K H --> K

---

6. Dataset Comparison and Recommendation

6.1 Comparison Table

Criterion	Porto Taxi Synthetic	Alibaba 2017	Alibaba 2018	Google Cluster v2
Forecasting suitability	Excellent — used and verified	Limited — 12h usage data only	Possible	Possible
Task-level records	None	batch_task.csv with job_id, start, end, cpu_req, mem_req	Yes, plus DAG dependencies	task_events.csv
Machine capacity	None	server_event.csv normalized CPU/mem/disk	Yes	machine_events.csv
CPU/Mem utilization	None	server_usage.csv at 60s intervals	Yes	resource_usage at 5min

Criterion	Porto Taxi Synthetic	Alibaba 2017	Alibaba 2018	Google Cluster v2
Task duration	None	Derived: end_time minus start_time	Yes	Yes
Missing data	None (synthetic)	Usage only for 12h; some null durations	Higher complexity	Obfuscated resource units
Offload simulation feasibility	Not feasible	Feasible with documented assumptions	Feasible	Feasible
Dataset size	~10 MB preprocessed	~1.5 GB compressed	~5 GB compressed	~40 GB
APSM pipeline compatibility	Already integrated	Separate pipeline required	Separate pipeline required	Separate pipeline required
License	Apache 2.0	Public research, free access	Public research	CC BY 4.0

6.2 Recommendation

> *[!IMPORTANT]* ***APSM experiments (E1, E2):*** *Porto Taxi synthetic trace — unchanged, no new data pipeline needed.* ***MARL experiments (E3, E4):*** *Alibaba Cluster Trace 2017 —* `batch_task.csv` *for task arrivals and durations,* `server_usage.csv` *for machine utilization profiles.*

Why Alibaba 2017 over 2018: Simpler schema, well-studied, no DAG task dependencies. 24-hour period maps cleanly to a single-day train/val/test split.

Why not Google Cluster v2: 40 GB download; resource units obfuscated; excessive preprocessing for a first MARL experiment.

Mandatory pre-implementation data validation:

- Confirm `batch_task.csv` columns `start_time` and `end_time` are non-null for >80% of records.
- Confirm `server_usage.csv` covers at least 12 consecutive hours.
- Confirm derived duration (`end_time − start_time`) is positive for >90% of records (negative values indicate trace logging artifacts, not real data).

Key assumption to document explicitly in thesis: Alibaba 2017 machines are datacenter nodes mapped synthetically to a k-NN edge topology. Machine capacities (normalized in the trace) are treated as dimensionless capacity units. This is a simulation assumption, not a real-world measurement.

---

7. Training Strategy: CTDE with MAPPO

7.1 Overview

Phase	Who observes	Who is updated
Offline training	Critic sees concatenated global state of all N nodes	Shared actor and critic updated jointly via PPO loss
Frozen deployment	Each actor sees only its local NodeObservation	No gradient updates; parameters frozen

7.2 MAPPO Configuration

- **Actor:** Shared MLP [64, 64] with ReLU. Parameters θ are identical across all N nodes (homogeneous).
- **Critic:** Separate MLP [128, 128, 64] with access to flattened global state. Discarded after training — never deployed.
- **Why shared parameters:** All nodes are homogeneous hardware in the Alibaba trace. Sharing reduces sample complexity by N× and is the standard choice in homogeneous MARL literature.
- **Training loop:** Collect trajectories from the task-execution simulator → compute GAE advantage with centralized critic → update actor and critic with clipped PPO loss.

7.3 Information Leakage Guard

The critic accesses global state only during offline Alibaba 2017 trace replay. At evaluation time, the frozen actor must access only fields available from local node state or bounded gossip context. This boundary must be enforced as a unit test (see Section 13).

---

8. State, Action, Reward, and Policy Design

8.1 State Space — NodeObservation (15-dimensional for k=3)

Feature	Source	Units	Normalization	Available at runtime
`forecast_load` $\hat{y}_i(t+1)$	LSTM inference	[0, 1]	None needed	Local
`apsm_surprise` $\varepsilon_i(t)$	node.last_surprise	[0, ∞)	Clip via running max	Local
`apsm_threshold` $\tau_i(t)$	node.last_threshold	[0, ∞)	Same running max	Local
`local_queue_depth` $q_i(t)$	task_env	integer	divide by Q_max	Local
`local_cpu_util` $u_i(t)$	task_env or trace	[0, 1]	Already normalized	Local
`local_mem_util` $m_i(t)$	task_env or trace	[0, 1]	Already normalized	Local
`task_cpu_req` c_t	current task	[0, 1]	divide by machine capacity	Local from task

Feature	Source	Units	Normalization	Available at runtime
`task_mem_req` r_t	current task	[0, 1]	divide by machine capacity	Local from task
`task_deadline_slack` d_t	current task	seconds	divide by D_max	Local from task
`neighbor_j_cpu_avail` (×k)	gossip-piggybacked	[0, 1]	None	Stale gossip — see warning
`neighbor_j_queue_depth` (×k)	gossip-piggybacked	integer	divide by Q_max	Stale gossip
`apsm_suppressed_flag`	node.consecutive_suppressions > 0	binary	None	Local

> *[!WARNING] Neighbor state features will be stale when APSM suppresses transmissions. This is not a bug — it is a research variable. Track `state_age_j` (timesteps since last gossip update per neighbor) and include it as an optional additional feature. The policy is trained to handle staleness; if H2 is falsified (staleness degrades performance significantly), that is a publishable finding.*

Total dimension: 9 + 2k = 15 for k=3.

8.2 Action Space (5 actions for k=3)

Index	Action	Validity constraint
0	Execute task locally	Always valid (fallback)
1	Offload to neighbor 0	Valid only if neighbor has sufficient CPU and mem capacity and is not FAILED
2	Offload to neighbor 1	Same
3	Offload to neighbor 2	Same
4	Defer task	Valid only if deadline_slack > min_defer_threshold

Action masking is applied before the policy softmax. If all offload actions are masked (all neighbors full or failed), the policy selects between local and defer only.

8.3 Reward Function

Per-step reward for node i after executing action a:

```
r_i(t) = − α · L_i(t)
         − β · I[deadline missed]
         − γ · I[offload failed] · c_transfer
         − δ · max(0, u_i(t) − u_thresh)
```

Term	Role	Search range for tuning
α · L_i(t)	Latency penalty (normalized)	α ∈ [0.3, 1.0]
β · I[deadline missed]	SLA violation — must dominate	β ∈ [5.0, 20.0]
γ · I[offload failed]	Failed offload penalty	γ ∈ [1.0, 5.0]
δ · overload	CPU saturation penalty	δ ∈ [0.5, 2.0]

Global training signal (LGRS adaptation):

During training only: `A_i = r_i_local + λ · r_global − V_baseline` where `r_global` = mean reward across all N nodes at this timestep. λ ∈ [0, 1] controls global influence. λ=0 reduces to pure local reward (used as ablation A2).

> *[!NOTE] Do not invent optimal λ or reward coefficients. Tune on the validation split (Alibaba 2017 hours 17–20). Fix values before running the test split (hours 20–24). Report a ±50% sensitivity analysis.*

---

9. Task Execution and Completion-Time Modeling

9.1 Approach: Trace-Calibrated Simulation

Use observed Alibaba 2017 records to calibrate the task simulator. Do not build a secondary ML execution-time predictor for the first experiment — it adds complexity without clear scientific benefit.

9.2 Task Completion Time Formula

For task τ assigned to node j:

```
T_completion(τ, j) = T_queue(j) + T_transfer(τ) + T_exec(τ, j)
```

- **T_queue(j):** Sum of remaining execution times of tasks ahead in j's queue.
- **T_transfer(τ):** 0 for local execution; sampled from [25, 35] simulation seconds for offloaded tasks (matching existing gossip transmission model).
- **T_exec(τ, j):** Sampled from a log-normal distribution fitted to Alibaba 2017 `batch_task.csv` durations, conditioned on task CPU request bucket (low / medium / high).

Required thesis statement: "Task execution durations are drawn from a log-normal distribution fitted to Alibaba 2017 batch task records (field: `end_time − start_time`). This is a calibrated simulation assumption; real heterogeneous hardware would produce different distributions."

9.3 Invalid Action Handling

When an offload is attempted to a node without sufficient capacity:

1. Return task to local queue (offload attempt fails silently).
2. Apply `failed_offload` reward penalty.
3. Log event as `OffloadRejectionEvent` in MARL metrics.

This models realistic rejection without crashing the simulation or requiring pre-coordination.

---

10. HRAC and LGRS Applicability

HRAC

Recommendation: Do not adopt HRAC for Phase 3.

The k=3 nearest-neighbor random geometric topology produces structurally homogeneous nodes. There are no natural roles (no coordinators, no storage-only nodes, no gateway nodes). Introducing role-specific policies would multiply hyperparameters by 2–3× with no clear benefit for the first experiment.

Decision gate for Phase 4: If Phase 3 shows that policy agents at high-degree graph positions consistently underperform relative to low-degree nodes, introduce HRAC as a Phase 4 ablation and compare.

LGRS

Recommendation: Adopt minimal LGRS — centralized critic, local+global reward blend, training-phase only.

The motivating problem from Yao et al. directly applies: pure local rewards lead to selfish offloading where all agents push to the same high-capacity neighbor, recreating the bottleneck that offloading was meant to solve. The global critic signal corrects coordination failures without requiring a permanent central server.

This is a meaningful adaptation and should be described as a design contribution in the thesis, contrasting it with Yao et al.'s permanent centralized reward service.

---

11. Proposed Module and File Structure

```
APSM-MTP/
├── src/                              ← EXISTING — do not modify
│   ├── gossiplearning/               ← EXISTING
│   ├── utils/                        ← EXISTING
│   ├── experiment_config.py          ← EXISTING
│   ├── run_all_comparisons.py        ← EXISTING
│   └── ...
│
├── marl/                             ← NEW top-level package
│   ├── __init__.py
│   ├── config.py                     ← Pydantic config for all MARL hyperparameters
│   ├── state_interface.py            ← NodeObservation dataclass + extractor function
│   ├── task_models.py                ← Task, Machine, Queue dataclasses
│   ├── task_env.py                   ← Discrete-event task-execution simulator
│   ├── policy.py                     ← Actor MLP (PyTorch)
│   ├── critic.py                     ← Centralized critic MLP (training only)
│   ├── trainer.py                    ← MAPPO training loop (CTDE)
│   ├── evaluator.py                  ← Frozen-policy evaluation runner
│   └── metrics.py                    ← MARL-specific metrics and logging
│
├── data/
│   ├── datasets/                     ← EXISTING Porto datasets
│   └── alibaba_2017/                 ← NEW (download separately)
│       ├── batch_task.csv
│       ├── server_usage.csv
│       └── server_event.csv
│
├── results/
│   ├── phase2_full_10n/              ← EXISTING
│   ├── phase2_full_50n/              ← EXISTING
│   └── phase3_marl/                  ← NEW
│       ├── e3_marl_only/
│       └── e4_apsm_marl/
│
├── marl_experiment_config.py         ← NEW central config for Phase 3
├── run_marl_training.py              ← NEW MARL training entry point
└── run_marl_evaluation.py            ← NEW frozen-policy evaluation entry point
```

Key constraint: No file in `src/gossiplearning/` or `src/utils/` is modified. The `marl/state_interface.py` reads APSM node state via read-only access to already-public Node attributes.

---

12. Detailed Data Flow and Edge Case Handling

12.1 Normal Operation (E4: APSM + MARL)

```
1. Task arrives at node i (from Alibaba 2017 replay or Poisson process)
   Task attributes: {cpu_req, mem_req, deadline, arrival_time}

2. state_interface.py constructs NodeObservation for node i:
   - forecast_load  = node._model.predict(last_X_window)[0][0]
   - apsm_surprise  = node.last_surprise
   - apsm_threshold = node.last_threshold
   - local_cpu_util, local_queue_depth from task_env
   - neighbor states from task_env.get_last_known_neighbor_states(i)

3. actor_policy.act(observation, action_mask) → action in {0..4}

4. task_env.execute_action(task, action, node_id=i):
   - action=0 (local):      enqueue at node i
   - action=1..3 (offload): check capacity; enqueue at j or reject and return to i
   - action=4 (defer):      re-queue with lower priority after slack seconds

5. task_env.step() advances discrete simulation time
   - task completes or deadline expires
   - reward r_i computed from latency, deadline status, overload

6. metrics.log_step(node_id=i, action, reward, task_status)
```

12.2 Edge Cases

Scenario	Handling
Forecast missing (LSTM not yet warmed up)	Use `forecast_load = 0.5` as neutral prior; log as missing
Neighbor state stale (APSM suppressed recent rounds)	Include `state_age_j` feature; policy is trained to discount stale info
Node fails during task execution	Task fails; agent receives full deadline-miss penalty
Destination lacks capacity (offload rejected)	Task returns to local queue; failed-offload penalty applied; event logged
All offload actions masked simultaneously	Force action=0 (local); log forced-local event
APSM suppresses model update	Gossip state freezes; MARL task env continues independently

Scenario	Handling
Task cannot meet deadline even locally	Log as unrescuable; do not apply penalty if all actions were masked

---

13. Experimental Design

13.1 Core Experiments and Ablations

ID	Name	APSM	MARL	Dataset	Purpose
E1	GL-Baseline	OFF	No	Porto	Existing — do not re-run
E2	APSM	ON	No	Porto	Existing — do not re-run
E3	MARL-Only	OFF	Yes	Alibaba 2017	MARL in isolation
E4	APSM + MARL	ON	Yes	Alibaba 2017	Combined system
A1	Reactive Heuristic	OFF	Threshold rule	Alibaba 2017	MARL lower bound
A2	MARL local reward	OFF	Yes, λ=0	Alibaba 2017	LGRS ablation
A3	MARL global reward	OFF	Yes, λ=1	Alibaba 2017	LGRS ablation
A4	Oracle Scheduler	ON	Omniscient	Alibaba 2017	Upper bound

> *[!IMPORTANT] E1 and E2 use Porto Taxi data. E3, E4, and all ablations use Alibaba 2017 data.* ***Do not compare MSE numbers (Porto) with latency numbers (Alibaba) in the same table.*** *Report in separate sections with clearly labeled dataset provenance.*

13.2 Scaling Configuration

Network sizes: N ∈ {10, 50} for Phase 3; N=100 pending feasibility profiling. k=3 neighbors. Each configuration: 5 random seeds; report mean ± std.

13.3 Metrics

For E1, E2 (existing): Avg MSE scaled and raw, packet reduction %, MSE delta vs GL-Baseline %.

For E3, E4, and ablations (new):

- Mean task completion latency (s) — primary metric
- 95th percentile task completion latency
- Deadline-miss rate (%)
- Task offload success rate (%)
- Per-node CPU utilization (mean and std across nodes)
- Overload events (CPU > 0.9 sustained)
- Gossip model-update traffic (bytes) — existing counter
- Task-transfer traffic (bytes) — new counter, separate from model traffic
- Total network overhead = gossip bytes + task-transfer bytes
- Policy reward convergence curve during training

- Run-to-run reward variance across 5 seeds

---

14. Research Novelty Assessment

14.1 Established Contributions (Not Novel as Standalone)

- LSTM time-series forecasting: standard
- Gossip Learning for decentralized FL: Hegedűs et al. 2019; Tundo et al. 2025
- MARL for resource management: Wang et al. 2021; Yao et al. 2025
- CTDE with MAPPO: Yu et al. 2022 (standard technique)

14.2 Novel Contributions

Phase 1/2 (confirmed original):

1. APSM semantic filter: pre-transmission gating for Gossip Learning via adaptive surprise scoring
2. Empirical validation of bandwidth invariance with <5% accuracy cost at 10 and 50 nodes

Phase 3/4 (proposed — novelty pending validation): 3. **APSM-state as MARL observation feature:** Using $\varepsilon(t)$ and $\tau(t)$ directly in the RL state vector. The policy learns to correlate communication confidence with resource decisions. Not present in existing literature. 4. **Interaction study between semantic gating and MARL state freshness:** When APSM suppresses gossip, neighbor states become stale. Quantifying this degradation and training policies robust to it is an original research question. 5. **Decentralized P2P MARL for task offloading without a central coordinator:** Existing MARL-based edge offloading assumes a master gateway. The k-NN gossip topology with no master is underexplored.

14.3 Falsifiable Hypotheses

H1: Forecast-driven MARL (E3) reduces mean task completion latency vs. reactive heuristic (A1) by a statistically significant margin ($p < 0.05$) under non-stationary workload.

H2: APSM filtering (E4 vs. E3) does not significantly degrade MARL task latency when state freshness features are included in the NodeObservation.

H3: LGRS global critic signal (A3) achieves lower cross-node CPU utilization variance than pure local reward (A2).

H4: APSM + MARL (E4) achieves a better bandwidth-vs-latency Pareto point than either mechanism alone.

---

15. Risk Register

Risk	Severity	Mitigation
Alibaba 2017 task durations contain nulls or negatives	High	Validate before any coding; filter; set minimum viable task count
Porto workload ≠ Alibaba cluster workload (incompatible comparison)	High	Keep experiments and result tables strictly separate by dataset

Risk	Severity	Mitigation
APSM state staleness makes MARL policy untrainable	High	Include state_age features; use A2 as control; negative result on H2 is publishable
Centralized critic leaks global state to actor at runtime	High	Actor module must never import critic module; enforce via unit test
MAPPO training instability	Medium	Use RLlib or known-stable MAPPO implementation; clip rewards to [-10, 0]; use entropy regularization
Simulated task durations do not reflect real hardware	Medium	Document explicitly in every results table caption as simulation assumption
N=100 task simulation is computationally prohibitive	Medium	Profile at N=10 and N=50 before committing to N=100; use vectorized environment
Reward coefficients overfit validation split	Medium	Fix coefficients after validation; run ±50% sensitivity analysis
HRAC introduced prematurely without architectural justification	Low	Enforce Phase 3 homogeneous-only policy; gate HRAC introduction on Phase 4 ablation results

---

16. Implementation Roadmap

> *[!IMPORTANT] This roadmap must not be executed until the blueprint is approved. Steps are ordered by dependency, not by calendar schedule.*

Pre-implementation gates:

- [ ] Alibaba 2017 data downloaded and duration field validated
- [ ] This blueprint approved by supervisor
- [ ] Thesis chapter outline updated to reflect Phase 3 scope
- [ ] All 6 unresolved questions in Section 18 answered

Ordered implementation steps:

1. `marl/task_models.py` — Task, Machine, Queue dataclasses; no ML, pure data structures
2. `marl/task_env.py` — Discrete-event task simulator driven by Alibaba 2017; validate standalone before adding MARL
3. `marl/state_interface.py` — NodeObservation dataclass; read-only access to gossiplearning.node.Node public attributes
4. `marl/metrics.py` — All MARL metric collectors; verify existing APSM metrics are unaffected
5. `marl/policy.py` — Actor MLP [64, 64]; no training yet
6. `marl/critic.py` — Centralized critic MLP [128, 128, 64]; training use only
7. `marl/trainer.py` — MAPPO training loop; train on Alibaba 2017 replay without gossip first
8. `marl_experiment_config.py` — Centralized Pydantic config for all Phase 3 parameters
9. `run_marl_training.py` — E3 entry point (MARL without APSM)
10. `run_marl_evaluation.py` — Frozen-policy evaluation for E3 and E4

11. E4 integration — connect APSM node state through state_interface to actor
12. Ablation runs — A1 heuristic, A2 local reward, A3 global reward, A4 oracle
13. Scaling — N=50 experiments; N=100 pending feasibility profiling

---

17. Acceptance Criteria

The architecture is approved for implementation when all of the following pass:

Criterion	How to verify
Alibaba 2017 batch_task.csv has >80% non-null positive durations	Manual data inspection script before any code
Porto APSM results are reproducible	Re-run E2 with seed=0; confirm MSE and packet counts match published results
NodeObservation interface requires zero changes to node.py	List all accessed Node attributes; confirm all are already public
Task simulator runs standalone without gossip simulator	Unit test: run task_env.py with synthetic task arrivals only
Reward masking prevents defer on near-deadline tasks	Analytical review of action masking logic
Alibaba 2017 train/val/test split is strictly chronological	Define explicit hour-based timestamps before implementation
Supervisor has reviewed and approved this blueprint	External sign-off documented

---

18. Unresolved Questions Requiring Your Decision

The following 6 questions cannot be resolved from the repository or this blueprint alone. They require explicit decisions from you (the candidate) before implementation:

1. **RL library:** RLlib (has native MAPPO, steeper learning curve), stable-baselines3 (no native MAPPO, requires wrapper), or implement from scratch? **Recommended:** RLlib for production-quality MAPPO.

2. **Simulator coupling:** Should the task-execution simulator run in a synchronized discrete-event loop alongside the gossip simulator, or as an independent loop with periodic state synchronization? **Recommendation:** Independent loops for Phase 3 to preserve APSM reproducibility; document the coupling model explicitly.

3. **Task arrival process:** Exact Alibaba 2017 trace timestamp replay, or a Poisson process parameterized by the trace arrival rate? **Recommendation:** Exact replay first; Poisson as sensitivity ablation.

4. **Forecast exposure:** The LSTM requires a 4×9 input window. At MARL decision time, should the interface reuse the last training batch window, or maintain a separate rolling inference buffer? **Recommendation:** Separate rolling buffer for cleanliness, implemented in `state_interface.py`.

5. **Offloading communication cost:** Model task transfer bytes as a function of task input size (not in Alibaba 2017 records), or assume a fixed per-task transfer cost? **If fixed:** what value? This needs a justified simulation assumption.

6. **Scaling target:** Should Phase 3 include N=100 MARL experiments, or stop at N=50 given the added training cost? This depends on available compute budget and thesis timeline.

---

19. Key Findings from Codebase Inspection

For completeness, the following are the most consequential findings that directly shape the proposed design:

1. **The LSTM forecast is entirely internal.** `node.py:perform_update()` → `train_model()` produces `val_loss` but not $\hat{y}(t+1)$ as a standalone output. The MARL state interface must call `node._model.predict(window)` at decision time, requiring a maintained input window buffer.

2. **The gossip simulator has no resource model.** No CPU, memory, queue, or task concept exists in any of the 14 files in `src/gossiplearning/`. The task-execution simulator is a completely new subsystem.

3. **`TRAIN_TIME = 5` is hardcoded.** This is an unresolved `TODO` in `event.py` line 284. The MARL layer should not depend on this value.

4. **The heartbeat default is 3, not 5.** The `config.py` default is 3; `experiment_config.py` overrides to 5. This discrepancy is harmless for existing experiments but must be documented so future configs set it explicitly.

5. **Porto datasets are fully synthetic.** The "Porto Taxi" branding is geographic — the workload values are AR(4) noise + smooth trend, not real GPS trip data. This is important when claiming dataset realism in the thesis.

6. **Public attributes sufficient for MARL interface.** `node.last_surprise`, `node.last_threshold`, `node.consecutive_suppressions`, `node.suppressed_count`, and `node._best_val_loss` are all accessible without any modification to `node.py`. Only the LSTM inference call requires maintaining an external input window buffer.